Ex. NO: 12 Date: 03/09/2024

End - End communication at transport layer

a& Implement 'Echo' client-server using TCP/UDP socket

b& Implement chat client-server using TCP/UDP sockets

Aim:

a& To implement echo client-server using TCP / UDP sockets.

b& To implement chat client-server using TCP / UDP sockets.

Algorithm:

PART A - Echo client server:

*. Implement an echo client - server using socket programming.

*. Use TCP / UDP sockets for communication between the client & server.

*. Establish client-server communication through the transport layer.

*. Exchange data / message between the client and server.

*. Record the input and output obtained during the communication.

PART B - chat client server:

*. Implement a chat client - server using socket programming.

*. Use TCP / UDP sockets for communication between the client and server.

*. Establish communication between the client and server.

*. Exchange messages between the client and server.

*. Observe the message received at both the client & server sides.

Input:

PART A:

Message / data given by the client for echo communication.

PART B:

Message exchanged by the client and server during chat communication.

~~Result~~:

X Thus, Echo client - server & chat client - server communication using TCP / UDP sockets were successfully implemented. X

TCP:

```
import socket
import threading
# ---- SERVER ----
def start_server():
    server = socket.socket(socket.AF_INET,
                           socket.SOCK_STREAM)

    server.bind(('127.0.0.1', 5000))
    server.listen(1)
    print("[SERVER] listening on 127.0.0.1:5000")

    conn, addr = server.accept()
    print(f"[server] connected by {addr}")

    while True:

        data = conn.recv(1024).decode()
        if not data or data.lower() == 'quit':
            print("[SERVER] received : {data}")
            print("[SERVER] connection closed by client.")
            break
        print(f"[SERVER] Received: {data}")
        conn.send(f"server received: {data}".encode())
    conn.close()
    server.close()

# ---- CLIENT ----

def start_client():
    client = socket.socket(socket.AF_INET, socket.
                           SOCK_STREAM)

    client.connect(('127.0.0.1', 5000))
    print("[CLIENT] connected to server")

    while True:
        msg = input("Enter message (or 'quit' to
                    exit): ")

        client.send(msg.encode())
```

```
        if msg.lower() == 'quit':
            break
        response = client.recv(1024).decode()
        print(f"[SERVER RESPONSE] {response}")
    client.close()

# ------ MAIN ------

if __name__ == "__main__":
    server_thread = threading.Thread(target=
                                     start_server,
                                     daemon=True)
    server_thread.start()
    start_client()
```

Output :

```
[SERVER] Listening on 127.0.0.1:5000 [CLIENT] connected
                                    to server
[SERVER] connected by ('127.0.0.1',53458) Enter
                        message (or 'quit' to exit):
 hello

[SERVER] Received : hello
[SERVER RESPONSE] server received : hello
Enter message (or 'quit' to exit):
- - - - -
[SERVER] Listening on 127.0.0.1:5000 [CLIENT] connected to server
[SERVER] connected by ('127.0.0.1', 60328) Enter message (or 'quit
                                                    to exit):
 Hi
[SERVER] Received Hi
[SERVER RESPONSE] server received : Hi
Enter message (or 'quit' to exit): quit
[SERVER] connection closed by client.
```

UDP:

```
import socket
import threading
def udp_server():
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    s.bind(('127.0.0.1', 12000))
    print("[SERVER] Running on 127.0.0.1:12000")
    while True:
        data, addr = s.recvfrom(1024)
        msg = data.decode()
        if msg.lower() == 'quit':
            print(f"[SERVER] client {addr} ended the
                                        connection.")
            break
        print(f"[SERVER] Received from {addr}: {msg}")
        s.sendto(data, addr)
    s.close()
    print("[SERVER] stopped.")
# ------ CLIENT ------
def udp_client():
    c = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    server = ('127.0.0.1', 12000)
    while True:
        msg = input("Enter message (or 'quit' to stop):")
        c.sendto(msg.encode(), server)
        if msg.lower() == 'quit':
            print("[CLIENT] closing connection...")
            break
        data, _ = c.recvfrom(1024)
        print("Echo from server:", data.decode())
    c.close()
    print("[CLIENT] stopped")
```

```
# ------ MAIN ------
if __name__ == "__main__":
    threading.Thread(target=udp_server,
                     daemon=True).start()
udp_client()
```

Output:

```
Enter message (or 'quit' to stop): [SERVER] Running on
                                   127.0.0.1:12000
hi
[SERVER] Received from ('127.0.0.1', 49670): hi
Echo from server: hi
Enter message (or 'quit' to stop): hello
[SERVER] Received from ('127.0.0.1', 49670): hello
Echo from server: hello
Enter message (or 'quit' to stop): how are you
[SERVER] Received from ('127.0.0.1', 49670): how are you
Echo from server: how are you
Enter message (or 'quit' to stop): quit
[CLIENT] closing connection... [SERVER] client
         ('127.0.0.1', 49670) ended the connection.
[CLIENT] stopped. [SERVER] stopped.
```

Result:

Thus, Echo client-server & chat client-server communication using TCP/UDP sockets were successfully implemented.